

# 1. Arquitecturas de Modernas de Software

La arquitectura de software moderna en 2026 ha dejado de ser una simple estructura de componentes para convertirse en ecosistemas dinámicos, inteligentes y altamente especializados. Hoy no existe una "mejor" arquitectura, sino una adecuada para cada problema. A continuación, desarrollo las arquitecturas predominantes, las herramientas que las sostienen y los lenguajes que dominan cada nicho.

## 1.1. Microservicios Orientados a Eventos (EDA)

Es la evolución natural de los microservicios tradicionales. En lugar de llamadas directas (HTTP/REST) que acoplan los servicios, los sistemas se comunican mediante la emisión y suscripción a eventos.

- **Concepto clave:** Desacoplamiento total. Un servicio de "Ventas" emite un evento `OrdenCreada`, y los servicios de "Inventario" y "Facturación" reaccionan a él de forma asíncrona.
- **Lenguajes**
  - **Go:** Por su gestión nativa de la concurrencia (*goroutines*) y baja latencia.
  - **Java (Spring Boot):** Sigue siendo el estándar corporativo por su robustez en el ecosistema de mensajería.
- **Herramientas**
  - **Orquestación:** Apache Kafka, RabbitMQ, Redpanda (una alternativa moderna y más rápida a Kafka).
  - **Observabilidad:** Prometheus, Grafana y Jaeger para rastreo de eventos.

## 1.2. Arquitectura Serverless y Cloud-Native

En 2026, *Serverless* ya no es solo "funciones aisladas", sino aplicaciones enteras que escalan a cero cuando no se usan, optimizando costos y sostenibilidad ("Green IT").

- **Concepto clave:** Pago por uso y abstracción total de la infraestructura. Se utiliza para APIs, procesamiento de datos en ráfagas y automatización.
- **Lenguajes:**
  - **TypeScript/Node.js:** El favorito por su rapidez de arranque (*cold start*) y ubicuidad.
  - **Python:** Dominante cuando la lógica serverless implica procesamiento de datos o IA.
  - **Rust:** Ganando terreno rápidamente gracias a su consumo mínimo de memoria y seguridad, ideal para funciones de alto rendimiento.
- **Herramientas**
  - **Proveedores:** AWS Lambda, Google Cloud Functions, Azure Functions.
  - **Frameworks:** Serverless Framework, AWS CDK, Pulumi (Infrastructure as Code).

## 1.3. IA Agéntica (Agentic Architectures)

Es la tendencia más disruptiva de 2026. No solo se trata de llamar a una API de IA, sino de diseñar el software como una red de "agentes" autónomos que colaboran para resolver tareas.

- **Concepto clave:** Integración de LLMs (Large Language Models) como el núcleo lógico del flujo de trabajo, capaces de usar "herramientas" (consultar bases de datos, enviar correos) de forma autónoma.
- **Lenguajes:**
  - **Python:** El lenguaje rey indiscutible de la IA (PyTorch, TensorFlow).
  - **Mojo:** Un lenguaje emergente que combina la sintaxis de Python con el rendimiento de C++.
- **Herramientas:**
  - **Orquestadores:** LangChain, LangGraph, CrewAI.
  - **Bases de Datos Vectoriales:** Pinecone, Milvus, Weaviate (para memoria semántica).

### 1.4.WebAssembly (WASM) en el Edge

WASM ha saltado del navegador hacia el servidor y el *Edge computing*. Permite ejecutar código a velocidades cercanas al metal en cualquier lugar.

- **Concepto clave:** Ejecutar aplicaciones complejas (edición de video, juegos, criptografía) directamente en el nodo de red más cercano al usuario para eliminar la latencia.
- **Lenguajes:**
  - **Rust:** El lenguaje mejor optimizado para compilar a WASM.
  - **C++:** Para sistemas legados de alto rendimiento que se migran al borde.
- **Herramientas:**
  - **Runtimes:** Wasmtime, Wasmer.
  - **Edge Platforms:** Cloudflare Workers, Vercel Edge Runtime.

### 1.5.Tabla Comparativa: Resumen de Tecnologías

| Arquitectura   | Enfoque                       | Lenguaje         | Herramienta        |
|----------------|-------------------------------|------------------|--------------------|
| Microservicios | Escalabilidad + Independencia | Go, Java         | Kubernetes, Kafka  |
| Serverless     | Agilidad + Costos             | Typescript, Rust | AWS lambda, Pulumi |
| IA Agéntica    | Autonomía + Inteligencia      | Python           | LangChain, Milvus  |
| WASM           | Latencia + Rendimiento        | Rust, C++        | Cloudflare Workers |

### 1.6.Nota sobre Seguridad: "AI-Native Defense"

Independientemente de la arquitectura, en 2026 la seguridad ya no es un "parche" (vulnerability patching), sino que se integra desde el diseño. Se utilizan **agentes de seguridad autónomos** que monitorizan el tráfico y aplican "parches virtuales" en tiempo real sin necesidad de reiniciar los sistemas.

## 2. Proyecto Curso: Evolucion de Vulnerabilidades.

Para lograr que la aplicación sea **asertiva** y muestre la **evolución** (y no solo una foto fija), la base de datos debe estar diseñada para manejar "series de tiempo". No basta con saber qué vulnerabilidades hay hoy; necesitamos saber cuándo aparecieron, cuándo desaparecieron y cuánto tiempo han estado activas.

Pueden utilizar **TimescaleDB** (basado en PostgreSQL) por su capacidad de manejar grandes volúmenes de datos históricos con un rendimiento superior a una base de datos relacional estándar.

### 2.1. Modelo de Datos: Entidades Principales

El diseño se basa en tres niveles: Infraestructura, Catálogo de Vulnerabilidades y Eventos de Detección.

#### A. Tabla **managers** (Configuración)

Almacena los diferentes Wazuh Managers a los que se conecta la app.

- `id`: UUID (PK)
- `nombre`: VARCHAR (Ej: "Manager-España", "Manager-USA")
- `api_url`: TEXT
- `api_key_vault_ref`: TEXT (Referencia al secreto en HashiCorp Vault)

#### B. Tabla **assets** (Servidores)

Información de los 500 servidores Ubuntu.

- `id`: UUID (PK)
- `wazuh_agent_id`: VARCHAR (ID que le asigna Wazuh)
- `hostname`: VARCHAR
- `os_version`: VARCHAR (Ej: "Ubuntu 22.04.3 LTS")
- `ip_address`: INET
- `manager_id`: UUID (FK a `managers`)

#### C. Tabla **vulnerability\_catalog** (Diccionario)

Evita duplicar descripciones largas de CVEs.

- `cve_id`: VARCHAR (PK) (Ej: "CVE-2023-1234")
- `severity`: ENUM ('Low', 'Medium', 'High', 'Critical')
- `description`: TEXT
- `cvss_score`: DECIMAL(3,1)

---

## 2.2.La "Hypertable" de Evolución: vulnerability\_detections

Esta es la tabla clave. En TimescaleDB, se convierte en una **hypertable** particionada por tiempo automáticamente.

| Columna         | Tipo        | Descripción                                                |
|-----------------|-------------|------------------------------------------------------------|
| timestamp       | TIMESTAMPTZ | Fecha/Hora exacta del escaneo del Wazuh (Clave de tiempo). |
| asset_id        | UUID        | Relación con el servidor afectado.                         |
| cve_id          | VARCHAR     | Relación con el Catálogo CVE                               |
| status          | ENUM        | “Detected”, “Resolved”, “Re-emerged”                       |
| package_name    | VARCHAR     | Nombre del paquete afectado.                               |
| package_version | VARCHAR     | Versión del paquete afectado.                              |

## 2.3.Lógica de "Detección de Evolución" (Stored Procedures)

Para que la visualización sea eficiente, la aplicación debe procesar los datos al entrar. No queremos que el frontend haga cálculos pesados.

1. **Detección de Nueva Vulnerabilidad:** Si llega un CVE para un servidor que no estaba en el último registro, se inserta con estado `Detected` y se marca la `first_seen_at` (timestamp de la detección).
2. **Mitigación Automática:** Si en el escaneo regular de Wazuh un CVE que estaba activo ya no aparece, el sistema inserta un registro automático con estado `Resolved`.
3. **Persistencia:** Si el CVE sigue apareciendo, se registra el evento para mantener el pulso en la gráfica de evolución.

## 2.4.Consultas Clave para el Dashboard

Para alimentar las gráficas "atractivas y asertivas", pueden usar consultas como estas:

- **Tendencia Semanal (Burn-down chart):**

SQL

```
SELECT time_bucket('1 week', timestamp) AS semana,
       count(*) AS total_vulnerabilidades
FROM vulnerability_detections
WHERE status = 'Detected'
GROUP BY semana ORDER BY semana;
```

- **Top 5 Servidores más vulnerables hoy:**

## SQL

```
SELECT hostname, count(cve_id) as total
FROM assets a
JOIN vulnerability_detections v ON a.id = v.asset_id
WHERE v.timestamp > NOW() - INTERVAL '7 days'
GROUP BY hostname ORDER BY total DESC LIMIT 5;
```

## 3. Escenario de prueba final

Para el escenario específico (500 servidores Ubuntu, múltiples Wazuh Managers, visualización histórica y filtrado avanzado), la elección de la arquitectura es crítica. No solo necesitas procesar datos, sino **correlacionar eventos en el tiempo** (evolución de vulnerabilidades). El siguiente es el análisis de ventajas y desventajas de aplicar cada arquitectura a este caso de uso de DevSecOps:

### 3.1. Microservicios Orientados a Eventos (EDA)

En esta arquitectura, deben generar o tener servicios independientes: uno para conectarse a las APIs de Wazuh, otro para procesar el histórico y otro para la visualización.

- **Ventajas:**
  - **Escalabilidad:** Si deciden monitorear 5,000 servidores en lugar de 500, solo se escalas el microservicio de "ingesta" (el que carag en la BD de la app la información proveniente de Wazuh)
  - **Resiliencia:** Si el Wazuh Manager 1 está caído, el sistema sigue procesando los datos del Manager 2 sin detenerse.
  - **Histórico Eficiente:** Pueden emitir un evento cada vez que una vulnerabilidad cambia de estado (Detected -> Mitigated), facilitando la creación de una línea de tiempo.
- **Desventajas:**
  - **Complejidad Operativa:** Requiere gestionar un orquestador de eventos.
  - **Consistencia Eventual:** La visualización podría tardar unos segundos en reflejar el estado real tras la petición de actualización.

### 3.2. Arquitectura Serverless y Cloud-Native

Ideal si las consultas a los Wazuh Managers se hacen de forma periódica (ej. cada lunes a las 8:00 AM) o bajo demanda del usuario.

- **Ventajas:**
  - **Costo-Eficiencia:** Si la aplicación solo se usa para reportes semanales, no pagas por servidores activos 24/7, solo por los minutos de ejecución del reporte.

- **Despliegue Rápido:** Puedes crear funciones separadas para cada Wazuh Manager y ejecutarlas en paralelo.
- **Desventajas:**
  - **Límites de Tiempo (Timeouts):** Si el volumen de datos de 500 servidores es muy alto, la API de Wazuh podría tardar más de lo que permite una función Lambda (normalmente 15 min).
  - **Cold Starts:** La primera vez que un usuario entre a filtrar datos tras horas de inactividad, la respuesta podría ser lenta.

### 3.3. Arquitecturas de IA Agéntica

Aquí, un agente inteligente actúa como intermediario entre el usuario y los datos crudos de Wazuh.

- **Ventajas:**
  - **Análisis Predictivo:** El agente no solo muestra la gráfica, sino que puede decir: *"He detectado que las versiones Ubuntu 18.04 están tardando un 40% más en ser parchadas que las 22.04"*.
  - **Interfaz Natural:** El usuario puede filtrar diciendo: *"Muéstrame solo vulnerabilidades críticas de red del último mes en servidores de producción"*.
- **Desventajas:**
  - **Costo de Tokens:** Procesar grandes volúmenes de registros de vulnerabilidades a través de un LLM puede ser costoso.
  - **Alucinaciones:** Riesgo de que la IA interprete erróneamente una métrica de seguridad si no hay "guardrails" estrictos.

### 3.4. WebAssembly (WASM) en el Edge / Cliente

En este caso, se usaría principalmente para la parte de **Visualización Atractiva y Eficiente** en el navegador del usuario.

- **Ventajas:**
  - **Rendimiento en el Filtrado:** Si descargas 100,000 registros de vulnerabilidades, filtrar y graficar eso con JavaScript puro puede congelar el navegador. WASM lo hace a velocidad de ejecución nativa (C++/Rust), permitiendo dashboards ultra fluidos.
  - **Privacidad:** El procesamiento de los datos para generar las gráficas ocurre localmente en la máquina del analista de seguridad.
- **Desventajas:**
  - **Curva de Aprendizaje:** Desarrollar componentes visuales en Rust/WASM es más complejo que usar librerías estándar como React o D3.js.
  - **Carga Inicial:** El módulo WASM puede ser pesado de descargar la primera vez.
  -

### 3.5. Resumen Comparativo

| Arquitectura   | Recomendación         | Por qué                                                                                                       |
|----------------|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Microservicios | Alta                  | Es la más robusta para manejar múltiples fuentes (Wazuh Managers) y mantener un histórico íntegro.            |
| Serverless     | Media                 | Excelente para ahorrar costos si el uso es esporádico (solo reportes semanales).                              |
| IA Agentica    | Baja o Complementaria | Úsarla solo como una capa superior para "chatear" con los datos, no como la base del sistema.                 |
| WASM           | Media                 | Úsarla específicamente para que el dashboard sea "atractivo y eficiente" al manejar miles de puntos de datos. |